Final Project: Pick a Project

Computer Science - Week 10

Oct 3, 2026 - Version 0.0.2

Please make sure to your UD **email** AND **name** is below. Unless your UD emails are included in this table, then you will not earn any points for this assignment when it is graded! **Every person should submit their own individual worksheet**.

Talk about each of the questions with a friend or our LLM (or both), but **make sure the words you enter below are yours** (with the obvious exceptions notwithstanding). Brainstorming is best done in full sentences, and the main goal here is brainstorming. It's worth reviewing the information in this series of pages in the Drafter documentation to gain a better understanding of the process.

Role	Name	Email
Developer (You)	Chasiti Bobet-Diaz	chasbd@udel.edu
Friend (optional)		

Part 1: The Goal & Prior Implementations

1. **Identify the Goal:** What specific goal are you trying to address? Think about your passions, hobbies, and academic interests. The latter are an excellent candidate to focus on. Some majors have suggestions listed below: these were provided by faculty in those departments. Other majors could consider a focus tied to a big final assignment in one of their recent courses in their major.
    - Majors in the **Electric & Computer Engineering** department may consider a steganography project that hides messages in images, and then retrieves those messages.
    - **Biomedical Engineering** majors may consider one of the following:
        - If you have started (or completed) the Cell/Tissue Design final project in BMEG100, design an organoid, which involves combining a cell type, biomaterial, and testing method. Have the website include a description and diagram of your design concept, a testing method, a needs statement, and an impact statement relevant to their organoid. Also include a link to a paper that supports your idea

and a paragraph describing how that paper supports your design concept.
        - Otherwise, create a website related to your BMEG100 Lab Report 1, *Cytoskeletal and Nuclear Staining Lab Report Guidelines*, linked here (University of Delaware account required to access).
    - **Chemistry** and **Chemical Engineering** majors may consider the project below.
        - Create an automated chemistry assistant that takes balanced reaction equations, starting reactant masses, and a harvested product amount to determine the limiting reactant, theoretical yield, and percent yield based on principles from Chemistry 2e (OpenStax) — Chapter 4: Chemical Reactions and Stoichiometry. To avoid manually compiling atomic mass tables or writing custom string tokenizers, use the chempy library to parse molecular formulas into exact molecular weights and verify reaction stoichiometry against compound data from the NIH PubChem Database. The program outputs an organized breakdown of limiting reagent status, product yields, and remaining masses of unreacted species. Also use the heat of the reaction to determine if it is exothermic or endothermic, and compute the heat flow per mole of the product.

Based on my Civil Engineering Major and my knowledge of Construction Site Procedure (from my Mother, a Construction Project Manager) I decided to make a website that could calculate the information she needs to fill out her time sheets on her construction site.

2. **Prior Implementations:** Search the internet. What existing tools or websites currently attempt to address it?

ClockShark, busybusy, WorkMax, ExakTime, SmartBarrel

3. **Your Implementation:** Briefly describe the legitimate purpose or functionality of your site. How would you want this web application to be used?

Users input the job, hours, and days worked of their work force and receive the information of their pay that they can use in their time sheets.

Part 2: The Input-Process-Output (IPO) Model

Think about the fundamental flow of your program.

1. **The Core Goal:** Every program has three main phases: taking input, processing data, and presenting output. In one sentence, what is the ultimate goal your program will accomplish for the user? Be specific and use CS1 concepts.

The program will collect worker, trade, hours-per-day, and days-worked inputs, store each timesheet entry as a dataclass instance, calculate total hours using loops and arithmetic operators, and display an organized construction-site timesheet summary.

2. Input: What specific information will the user need to provide to your program? (e.g., a city name, a numerical budget, a list of hobbies).

The user will enter the worker's name in a TestBox, choose the worker's trade from a SelectBox, enter hours worked per day in a numerical TextBox, and enter the number of days worked in another numerical TextBox. The user may also enter a project or job-site name and the timesheet period. Each entry represents one worker's time for that period.

3. Processing: At a high level, how will the computer manipulate or evaluate that data? (e.g., calculate the average, filter out options that are too expensive, compare the input against a database).

The program will check that the worker's name is not blank, a trade has been selected, hours are greater than zero, and days worked is a positive whole number. It will create a TimesheetEntry object for each valid submission and add it to a list in the Drafter State. A loop will go through the list to calculate each entry's total hours, the total hours for the site, total hours for each trade, and the total pay for each worker. If information is invalid, the program will display a helpful message instead of adding the entry.

4. Output: What will the user get back? (e.g., a recommendation, a calculated safety factor, a formatted schedule).

The user will see each worker's name, trade, hours per day, days worked, calculated total hours, and total pay for that time period. A summary page will display total labor hours for the site and total money spent for each trade, and totals for each trade. The user will also be able to review all entries and remove an incorrect entry. These results are record of hours entered, not an official payroll calculation.

Part 3: Data Abstraction

Identify the "nouns" of your problem.

1. **Representing Reality:** Abstraction is the process of representing the real world inside a computer by ignoring unnecessary details. What are the core "things" (nouns) your program needs to track? (e.g., *If building a recipe app, the nouns are "Recipe" and "Ingredient"*).

The main nouns are TimesheetEntry, TradeSummary, and SiteTimesheet. A TimesheetEntry represents one worker's recorded time for a timesheet period. A TradeSummary represents the combined hours for a particular trade. A SiteTimesheet represents the collection of entries for one construction site or project.

2. **Identifying Properties:** For the most important "thing" you listed above, what 3 to 4 specific details do you actually need to solve your problem, and what details will you abstract away?
    - *Details I need:* (e.g., *Ingredient name, weight, cost*)
    - *Details I will ignore:* (e.g., *Ingredient color, brand, expiration date*)

Details I Need: Worker name, Trade, Hours worked, days worked, calculated total hours, site name, timesheet period
Details I will Ignore: Home Address, Phone Number, Social, Deductions/Taxes/Benefits, GPS location, clock-in/out times, break tracking, shift scheduling, detailed tasks, equipment usage, and productivity measurements.

Part 4: Challenges, AI Strategy, & Goals

1. **Difficulties:** What parts of this project seem the most difficult or technically complex?

The most challenging parts may be designing the dataclasses, keeping the list of timesheet entries in the Drafter State, and making sure each route updates the correct state fields. I will need to validate numeric inputs so black, negative, or nonnumeric values do not cause errors. Another challenge will be calculator totals for the whole site and for each trade without counting entries incorrectly. I will test the add, view, summary and remove-entry routes.

2. **Coding Strategy:** If you will be using an LLM to help code, how will you ensure you understand the generated code and avoid "hallucinations" or "shallow

learning" (copying without understanding)? If you won't be using an LLM to help code, this project might take about 4 hours a week to complete, so list each of the days and times in which you work on the project.

I will use an LLM as a brainstorming and explanation resource, but will not blindly copy code. I will build the program in small parts and make sure I can explain what each function does, what information it receives, and how it changes the State. I will compare suggestions with the course notes, Bakery textbook, and Drafter documentation. I will test helper functions with normal, boundary, and invalid values. If an LLM suggests a technique or library not covered in class, I will check whether it is allowed and document it in the code. I will save the complete AI conversation in my GitHub repository as required.

3. **Goals:** What is your ultimate goal for this project? (e.g., *Impress the graders with a creative UI, build something I can use in my other classes, learn Drafter deeply*).

My goal is to create a useful, easy-to-understand website that connects my computer science coursework to civil engineering and construction management. I want to display my understanding. Additionally, I'm hoping that this could actually be used, and I will have my mom test it out for me since she is in the field.

Part 5: Research & Brainstorming Conversation

Have a conversation with a Generative AI tool (like ChatGPT or Claude) OR a friend about your project idea. Do not ask for code. Instead, use the AI/friend as a sounding board to research your problem.

1. **The Initial Prompt/Question:** Ask the AI or your friend to help you brainstorm. (e.g., *"I want to build a simple web tool to help students track their study hours. What are 3 common problems students face when tracking their time, and what are 3 simple features a basic tool should have?"*). Paste your prompt below:

I am a civil engineering major creating a beginner Python website with the Drafter library. I want to make a construction site timesheet tracker. A user should enter a worker's name, trade, hours worked per day, and number of days worked. What useful features should a simple version have, what calculations should it perform, and what existing tools are used to track construction worker time? Please keep the suggestions realistic for an introductory programming project and do not provide code.

2. **The Response:** Briefly summarize the best ideas the AI or your friend suggested. What existing solutions or prior apps did they mention?

Building a construction site timesheet tracker as an introductory Python project using the Drafter library is a fantastic way to bridge civil engineering concepts with software development. In construction, labor tracking is one of the most vital components of project management since labor often accounts for a major portion of a project's budget.

Realistic and helpful features, calculations, and industry tools for a beginner-friendly project include the following:

1. Useful Features for a Simple Version

Given the inputs you specified (worker's name, trade, hours per day, and number of days worked), a beginner web app can incorporate these practical features:

- **Worker Directory / Summary View:** A clean table or list that displays all submitted worker entries so a site superintendent or project manager can see everyone on-site at a glance.
- **Trade Filtering or Categorization:** The ability to sort or filter workers by their trade (e.g., Carpenter, Electrician, Laborer, Operator) to see how many labor resources are allocated to specific crafts.
- **Input Validation & Error Handling:** Simple checks to prevent a user from submitting negative numbers for hours or days, or leaving text fields blank.
- **Reset / Clear Functionality:** A way to clear individual entries or reset the active timesheet session.
- **Persistent Session or Summary Totals:** Displaying aggregate metrics at the top of the page, such as total workforce headcount and grand total hours logged for the entire site.

2. Essential Calculations to Perform

Beyond simply multiplying hours by days, a construction timesheet tracker should perform calculations that reflect real-world civil and construction engineering workflows:

- **Total Individual Hours:** Multiplying the *hours worked per day* by the *number of days worked* to find the total gross hours for an individual worker.
- **Estimated Gross Pay (by Trade/Hourly Rate):** Associating a standard base hourly wage depending on the trade (e.g., a master electrician typically commands a different hourly rate than a general laborer) and calculating total estimated labor cost per worker.
- **Overtime Estimation:** A standard work week in construction is often capped at 40 hours before overtime kicks in (typically paid at 1.5x, known as "time-and-a-half"). Your app can evaluate if a worker's total hours exceed a standard threshold and separate regular hours from overtime hours.

- **Aggregated Labor Cost:** Summing up the total labor expenditures across all workers to show project managers what the current payroll liability is for that tracking period.

3. Existing Tools Used in the Industry

In professional practice, construction companies have moved far beyond paper cards, though the underlying logic remains the same. Real-world tools include:

- **Specialized Construction Time-Tracking Software:** Tools like **ClockShark**, **Raken**, and **ExakTime (Arcoro)** are built specifically for the field. They often include mobile app clock-ins, GPS tracking, and geofencing to ensure workers are physically on the job site.
- **Enterprise Heavy Construction Software:** Platforms like **HCSS (HeavyJob)** or **Contractor Foreman** are widely used by civil engineering and heavy highway contractors. They link time cards directly to specific "cost codes" (standardized numerical codes used in estimating and bidding, like earthwork vs. concrete pouring).
- **General Spreadsheets (Excel / Google Sheets):** Despite dedicated software, many small-to-midsize subcontractors still rely heavily on customized Excel templates because they offer flexible calculation layouts similar to what you are building in Python.

Focusing your project on clear data organization, distinct trade wage logic, and overtime thresholds will give you a solid foundation that mirrors how real construction management software evaluates field productivity.

3. **The Pushback:** You must reply to the AI/friend to narrow the scope when it's over-ambitious. Pick the most complex feature they suggested and explain why it is too complicated for an introductory programming project. Ask them for a simpler alternative. *Paste your response here:*

GPS tracking, automatic clock-in and clock-out, payroll calculations, overtime rules, and employee scheduling are too complicated for my introductory Python project. Can you narrow the idea to a basic timesheet where I manually enter each worker's name, trade, hours per day, and days worked? I only need the program to calculate total hours and show summaries by worker and trade. The idea is to manually enter timesheet information and use basic arithmetic, lists, loops, and conditionals to organize it. The website should not connect to payroll systems, track worker locations, or determine overtime.

4. **Reflection:** Based on this conversation, how has your project idea changed or become more focused?

My project became more focused by limiting it to recording and summarizing time instead of managing an entire construction workforce. The final website will collect four main pieces of information for each entry and calculate total hours. It will also show site wide and trade-base summaries. This is realistic for an introductory programming course and relevant to civil engineering because construction projects need organized labor records.

Part 6: Resource Planning

1. What resources will you use if you get stuck?

I will use the course notes, Bakery textbook, Drafter documentation, class examples, officer hours, and help from my TA or instructor. I will also review documentation for construction time-tracking products to understand common timesheet information. I can use AI to explain error messages or help me reason through a programming problem, but I will verify suggestions using course materials and test the code myself.

2. If your LLM provides code that causes a Drafter or Bakery error, or if you run into a problem during deployment, what is your step-by-step debugging plan to resolve it?

1. Read the complete error message and identify the file, function, and line where the problem occurs
2. Check syntax, indentation, spelling, missing parentheses, and variable names
3. Test the total-hours calculation separately with values that can be checked by hand
4. Test input validation with blank names, missing trades, nonnumeric hours, zero, and negative numbers.
5. Check that adding an entry changes the State list and that viewing or removing entries works correctly
6. Test summary calculations with multiple workers in the same trade and workers in different trades
7. Run unit tests for helper functions and routes, ten test complete website
8. If deployment fails, review deployment instructions and error logs, check required files and imports, and ask the TA or instruction for help if needed

Part 7: LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously,

these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

Conversation Log: Construction Project Management Timesheet Websites

Date: October 30, 2026

User Prompt 1

I am a civil engineering major creating a beginner Python website with the Drafter library. I want to make a construction site timesheet tracker. A user should enter a worker's name, trade, hours worked per day, and number of days worked. What useful features should a simple version have, what calculations should it perform, and what existing tools are used to track construction worker time? Please keep the suggestions realistic for an introductory programming project and do not provide code.

Assistant Response 1

Building a construction site timesheet tracker as an introductory Python project using the Drafter library is a fantastic way to bridge civil engineering concepts with software development. In construction, labor tracking is one of the most vital components of project management since labor often accounts for a major portion of a project's budget.

Realistic and helpful features, calculations, and industry tools for a beginner-friendly project include the following:

1. Useful Features for a Simple Version

Given the inputs you specified (worker's name, trade, hours per day, and number of days worked), a beginner web app can incorporate these practical features:

- **Worker Directory / Summary View:** A clean table or list that displays all submitted worker entries so a site superintendent or project manager can see everyone on-site at a glance.
- **Trade Filtering or Categorization:** The ability to sort or filter workers by their trade (e.g., Carpenter, Electrician, Laborer, Operator) to see how many labor resources are allocated to specific crafts.
- **Input Validation & Error Handling:** Simple checks to prevent a user from submitting negative numbers for hours or days, or leaving text fields blank.
- **Reset / Clear Functionality:** A way to clear individual entries or reset the active

timesheet session.
- **Persistent Session or Summary Totals:** Displaying aggregate metrics at the top of the page, such as total workforce headcount and grand total hours logged for the entire site.

2. Essential Calculations to Perform

Beyond simply multiplying hours by days, a construction timesheet tracker should perform calculations that reflect real-world civil and construction engineering workflows:

- **Total Individual Hours:** Multiplying the *hours worked per day* by the *number of days worked* to find the total gross hours for an individual worker.
- **Estimated Gross Pay (by Trade/Hourly Rate):** Associating a standard base hourly wage depending on the trade (e.g., a master electrician typically commands a different hourly rate than a general laborer) and calculating total estimated labor cost per worker.
- **Overtime Estimation:** A standard work week in construction is often capped at 40 hours before overtime kicks in (typically paid at 1.5x, known as "time-and-a-half"). Your app can evaluate if a worker's total hours exceed a standard threshold and separate regular hours from overtime hours.
- **Aggregated Labor Cost:** Summing up the total labor expenditures across all workers to show project managers what the current payroll liability is for that tracking period.

3. Existing Tools Used in the Industry

In professional practice, construction companies have moved far beyond paper cards, though the underlying logic remains the same. Real-world tools include:

- **Specialized Construction Time-Tracking Software:** Tools like **ClockShark**, **Raken**, and **ExakTime (Arcoro)** are built specifically for the field. They often include mobile app clock-ins, GPS tracking, and geofencing to ensure workers are physically on the job site.
- **Enterprise Heavy Construction Software:** Platforms like **HCSS (HeavyJob)** or **Contractor Foreman** are widely used by civil engineering and heavy highway contractors. They link time cards directly to specific "cost codes" (standardized numerical codes used in estimating and bidding, like earthwork vs. concrete pouring).
- **General Spreadsheets (Excel / Google Sheets):** Despite dedicated software, many small-to-midsize subcontractors still rely heavily on customized Excel templates because they offer flexible calculation layouts similar to what you are building in Python.

Focusing your project on clear data organization, distinct trade wage logic, and overtime thresholds will give you a solid foundation that mirrors how real construction management software evaluates field productivity.

User Prompt 2

GPS tracking, automatic clock-in and clock-out, payroll calculations, overtime rules, and employee scheduling are too complicated for my introductory Python project. Can you narrow the idea to a basic timesheet where I manually enter each worker's name, trade, hours per day, and days worked? I only need the program to calculate total hours and show summaries by worker and trade. The idea is to manually enter timesheet information and use basic arithmetic, lists, loops, and conditionals to organize it. The website should not connect to payroll systems, track worker locations, or determine overtime.

Assistant Response 2

Narrowing your project scope to a clean, foundational Minimum Viable Product (MVP) is a smart approach for an introductory programming class. Focusing strictly on manual entry and core arithmetic allows you to master the fundamentals of Python and the Drafter library without getting bogged down by complex business logic.

A streamlined, beginner-friendly specification for your manual timesheet tracker includes the following components:

1. Core Features for a Basic Manual Tracker

With a focus on manual entry and straightforward data organization, your Drafter web application can feature:

- **Simple Data Entry Form:** A form with fields for worker name, trade (e.g., text input or a dropdown), hours worked per day, and number of days worked.
- **Master Timesheet Log:** A running table that displays every entry submitted during the active session, showing each worker's individual details right as they are added.
- **Worker Summary View:** A clean breakdown showing each individual worker alongside their cumulative total hours.
- **Trade Summary View:** A grouped breakdown showing each trade category (e.g., Laborer, Carpenter, Operator) and the grand total of hours logged across all workers in that trade.

2. Simple Calculations to Perform

Without payroll integrations or overtime rules, your math remains strictly focused on basic

arithmetic:

- **Individual Total Hours:**
  - $TotalHours = HoursperDay \times DaysWorked$
- **Trade Subtotals:** Summing the individual total hours for all workers sharing the same trade designation.
- **Site Grand Total:** Summing the total hours of every worker on the site to give a single aggregate number for total labor hours expended.

3. How Core Python Concepts Will Power Your App

Your project will naturally map directly to the foundational programming concepts you are learning:

- **Lists and Data Structures:** You can store each submitted timesheet entry (using a dictionary or parallel lists) so that the app remembers multiple workers across form submissions.
- **Loops (for / while):** You will use loops to iterate through your list of worker entries to calculate summary totals and dynamically render the rows in your summary tables.
- **Conditionals (if / else):** Conditionals will help you categorize workers by trade (e.g., checking if a worker's trade matches "Carpenter" to group their hours together) or validating that text fields aren't accidentally left blank.

This refined scope gives you a robust, fully functional civil engineering application that highlights essential programming logic while remaining entirely manageable for an introductory project.

Separate Conversation

1. User Query

"What are some websites that help construction project managers calculate time sheets"

2. Assistant Response

Construction project managers rely on dedicated digital time-tracking tools to replace manual paper sheets, auto-calculate hours and overtime, allocate costs to specific job

codes, and sync directly with payroll software.

Specialized Construction Time Tracking Tools

Website / Tool	Best Used For	Key Features
ClockShark	Small to mid-size subcontractors & trade crews	GPS geofencing, drag-and-drop worker scheduling, cost-code tracking, and direct integration with QuickBooks and ADP.
busybusy	Heavy civil, equipment operators, and sub-crews	Tracks machine/equipment utilization alongside labor hours, operates offline for remote job sites, and handles safety sign-offs.
WorkMax	Mid-to-large general contractors & enterprise teams	Built for complex multi-state prevailing wage calculations, certified payroll, high-volume cost coding, and enterprise ERP integration.
ExakTime	Field crews needing rugged options	Field-ready mobile app, physical rugged time clocks (Jobclock), automated overtime alerts, and photo ID clock-in verification to stop "buddy punching."
SmartBarrel	High-occupancy jobsites & hardware-first logging	Provides solar-powered, weather-resistant physical kiosk boxes for jobsites with facial recognition clock-in; no worker app required.

All-in-One Construction Management Platforms

If you are already managing project schedules, RFIs, and submittals, these platforms include built-in field timesheet calculation features:

- **Raken:** Combines daily reporting with field time tracking. Foremen can log crew hours, assign cost codes, and submit daily logs straight from their phones.
- **Buildertrend:** Popular among residential builders and remodelers; includes an integrated mobile clock-in/clock-out feature linked directly to project estimates and budgets.
- **Procore:** Enterprise construction management platform with a dedicated **Timecard** module that links labor cost data directly to real-time project budget tracking.

Free & Budget-Friendly Options

- **Jibble:** Free tier includes GPS tracking, facial recognition kiosk mode, automated break tracking, and overtime calculation.
- **Clockify:** Offers a comprehensive free plan with custom project tagging, time tracking, and exportable timesheets for small teams.

Part 8: Submit

In Google docs, with this document open, please click on *File* > *Download* > *Microsoft Word (.docx)*, and make sure you put that download somewhere you can find it again. Then, upload it to the Canvas assignment about selecting or brainstorming your final project.

When you're done with this assignment, review this page of the Drafter documentation to better prepare for the next step.